HotelPro

- Divesh Jaiswal

1. Project Overview

HotelPro is a hotel room discovery and guest booking application. The supplied screens show account access, date-based room search, room details and prices, booking records, and payment-status feedback. This report documents those visible capabilities and proposes a technical architecture and data model for the project.

Architecture basis: the repository layout supplied for this report identifies a React client and a separate server organized into config, models, controllers, routes, middleware, and utils. It also names an Axios API service and AuthContext. The database engine and the detailed behavior of each route still need confirmation.

2. Technical Architecture

HotelPro is organized as a client-server application. The React client contains pages and reusable components, keeps authentication state in AuthContext, and calls the server through services/api.js using Axios. On the server, routes connect requests to middleware and controllers; controllers use models and utilities, while config/db.js handles database setup.

Figure 1. HotelPro client and server request path

- Client: React pages include Login, Rooms, MyBookings, and AdminDashboard; components include shared navigation, room cards, and booking forms. AuthContext stores client-side authentication state.
- API communication: client/src/services/api.js uses Axios to send requests to the server. Server routes dispatch requests to controllers, with auth.js, role.js, and errorHandler.js middleware supporting authentication, authorization, and consistent error handling.
- Server logic: controllers coordinate request handling with models for User, Room, Booking, and Payment; utils contains shared helper logic. config/db.js and environment configuration establish the database connection. The database engine and schema details were not included in the supplied tree.
- Payment: the Payment model and guest UI indicate payment tracking. The folder tree does not identify an external payment provider or whether the demo payment is simulated.

3. Repository Structure

The supplied project layout separates the React interface from the server-side application. The tree below records the structure provided for this report; it does not imply that every listed page or operation has been fully tested.

```
hotelpro/
├── server/
│   ├── config/        (db.js, env)
│   ├── models/        (User, Room, Booking, Payment)
│   ├── controllers/
│   ├── routes/
│   ├── middleware/    (auth.js, role.js, errorHandler.js)
│   ├── utils/
│   └── server.js
└── client/
    └── src/
        ├── components/  (Navbar, RoomCard, BookingForm...)
        ├── pages/       (Login, Rooms, MyBookings, AdminDashboard...)
        ├── context/     (AuthContext)
        ├── services/    (api.js using axios)
        └── App.jsx
```

4. ER Diagram

The supplied tree confirms User, Room, Booking, and Payment model modules. The diagram below shows a logical relationship proposal; the actual fields, keys, and constraints must be checked in those model files.

Figure 2. Proposed HotelPro entity relationships

Entity	Purpose and key fields
User	Stores account identity and role. PK user_id; full name, unique email, phone, password_hash, role.
Room	Stores bookable inventory. PK room_id; room number, type, capacity, nightly rate, amenities, description.
Booking	Connects a guest to a room for a date range. PK booking_id; FK user_id, FK room_id; check-in/out, guest count, total, status.
Payment	Stores payment method and outcome. PK payment_id; FK booking_id; method, amount, status, transaction reference, paid_at.

Relationship rules: one user can have many bookings; one room can appear in many bookings over time; each booking belongs to one user and one room. A booking can have zero or more payment attempts, allowing unpaid reservations or retries. Availability should be calculated from overlapping bookings for the requested dates.

5. Features

Feature	What the screens show	Status
Account access	Login and registration pages; name, email, phone, and password fields.	Pages and forms visible; server-side behavior not verified.
Room search	Check-in/out dates, guest count, search action, room cards, capacity, amenities, nightly rate, and stay total.	Visible in supplied screens.
Room booking	Room cards provide a booking action; booking records appear in MyBookings.	Visible across screens; controller validation not verified.
Booking management	Booking list shows room, dates, amount, confirmed/payment state, and cancellation action.	Visible; cancellation result not shown.
Payment status	Payment method selector, Pay action, success notice, and paid state.	Visible; provider and live/simulated behavior unknown.
Admin dashboard	AdminDashboard page is named in client/src/pages; role middleware is also present.	Structure indicates admin UI and role checks; specific admin actions need source-code confirmation.

6. Roles and Responsibilities

Role	Responsibilities	Evidence / boundary
Guest	Access account; search dates and guest count; compare rooms and rates; place and review bookings; select payment method; request cancellation where supported.	Guest screens supplied.
Administrator	Use AdminDashboard for the operations implemented for authorized admin accounts. Maintain only the inventory, booking, or user functions exposed by the actual page and API.	AdminDashboard and role middleware are named in the structure; exact permissions and actions are not specified.
Application system	Authenticate requests; enforce role checks; validate input; calculate availability and totals; update booking/payment status; return consistent errors.	Middleware, models, controllers, and error handler are named; behavior should be confirmed in source.

7. User Flow

The user journey starts with account access. The project structure names both guest-facing pages and an AdminDashboard, while auth.js and role.js indicate authentication and role checks. The diagram shows the guest path from the supplied screenshots and a separate administrator entry path; the admin actions depend on the actual routes and controllers.

Figure 3. Guest and administrator entry paths

8. MVC Pattern

The supplied folders map cleanly to an MVC-style design. The React client provides the Views, server/models contains the Models, and server/controllers contains the Controllers. Routes dispatch HTTP requests, while middleware handles authentication, role checks, and errors. This is a client-server MVC mapping; the actual controller and model behavior depends on the source files.

MVC part	HotelPro mapping	Example responsibility
Model	server/models: User, Room, Booking, Payment.	Represent domain data and persistence; exact fields and database operations are defined in each model.
View	client/src/pages and client/src/components; AdminDashboard is a named page.	Render login, room search, booking views, shared components, and admin UI.
Controller	server/controllers.	Handle API request logic and coordinate model/service operations.
Request dispatch	server/routes.	Map API endpoints to controller actions.
Middleware	server/middleware: auth.js, role.js, errorHandler.js.	Authenticate, authorize by role, and handle errors before or around controller work.

Example request: the Rooms page sends a search through services/api.js and Axios; a server route dispatches it through authentication or role middleware where required; a controller validates the request and queries the Room model; the server returns matching rooms for the React view to render.

9. Implementation Verification Checklist

- ☐ Document the actual API endpoints and the controller action mapped to each route.
- ☐ Confirm the specific admin operations and permissions enforced by role.js and AdminDashboard.
- ☐ Confirm whether payment is simulated or provider-connected and how transaction references are stored.
- ☐ Verify that cancellation and payment actions update the persisted Booking and Payment records.
- ☐ Fill in the student, course, institution, and project-guide details on the cover page.
- ☐ Confirm the database engine, schema fields, key constraints, and room-availability query in server/config/db.js and server/models.

Use the source files to complete implementation-specific details that are not visible in the directory tree.

Conclusion

The supplied screens and folder layout establish a guest booking interface and a separate administrator page within a client-server MVC structure. The ER relationships and detailed admin operations should be checked against the model, route, and controller files before submission.